equilibrium

Kanon M0 Addendum A: LEZ v0.3

What the cost baseline looks like on the fee market

v2.0
2026-10-07

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

Logos
RFP-020

TABLE OF CONTENTS

Scope **3**
1. The answer in one page **4**
1.1 Disposition of M0′s recommendations 5
2. Recovery: the delivered backend **6**
3. The round: multi-point packages **7**
4. The chain model M0 lacked **8**
4.1 The per-block cap 8
4.2 Storage gas never binds 9
4.3 One recommendation revised: M0 §5.1′s keccak256 choice 9
5. Caveats, and asks **10**
6. Reproducing this **11**

Author: Equilibrium Group Oy
Status: Addendum to a milestone deliverable
Created: 2026-10-05
Label: zkvm-cost-baseline
Client: Logos
Target: RFP-020 milestone M0, task M0-12 — extends the M0 report of 2026-08-11

Scope

This is an addendum to the M0 report of 2026-08-11, keyed to its sections, and assumes it is read alongside. M0 measured against LEZ **v0.2.1**; the testnet runs **v0.3.0**, which replaced the program model and introduced a fee market. Logos asked whether the report still holds.

It does. Every M0 figure reproduces exactly under the frozen pins, the cryptography is unchanged to within ten cycles on 1.7 million, and each recommendation's disposition is one line in section 1. The addendum states two things per section: what M0 said, and the delivered state. A recovery costs **109,502 cycles** against the 568,350 of the path M0 priced (section 2), and the five-feed round settles in **12.9% of a block** against 155% for five single-point updates on M0′s path (section 3).

The v0.3 figures for M0′s path come from a port of `m0/lez-probe/` — same workload byte for byte, held honest by the guardrail `the_cryptography_did_not_move_between_lez_generations`; `m0/cost-baseline/` links no LEZ and is unaffected by construction. The delivered-product figures come from tests on the product repository (section 6). The full working, including the intermediate measurements this addendum deliberately omits, is `m0/v0.3_rerun/FINDINGS.md`.

1. The answer in one page

M0′s figures reproduce exactly on LEZ v0.3: the update M0 §1 priced at 1,906,737 cycles runs at 1,766,937 gas, the cryptography inside it unchanged to within ten cycles on 1.7 million. What changed is the chain underneath — v0.3 bills gas as cycles, one for one, against a 10,000,000 limit that also caps the whole block (section 4) — and the product on top: a recovery costs 109,502 cycles against the 568,350 of the path M0 priced (section 2), a 3-signer update costs 411,376 gas, and the five-feed round settles in 12.9% of a block where five single-point updates on M0′s path need 155% (section 3).

The delivered round — five feeds at the 3-of-5 threshold, every delivered optimization — costs 12.9% of a block's execution budget and 0.44% of its storage budget. On both of v0.3′s fee markets:

market	the round	share of the block cap	share of the fee target
execution (cap 10,000,000, target 5,000,000)	≈1,290,000 gas	**12.9%**	25.8%
storage (cap 1,000,000, target 500,000)	4,446 gas	**0.44%**	0.89%

Neither market is near binding, and execution remains the only one worth watching (sections 3 and 4).

M0 §1′s table, at three signers, on both chains:

component	at M0 (v0.2.1)	v0.3.0, same code
LEZ framework floor	39,103	**14,038**
LEZ instruction handling, 3 packages	119,187	**4,442**
3 × keccak256 over 77 bytes, software	51,956	51,956
3 × secp256k1 recovery and signature parse	1,696,491	**1,696,501**
whole update	**1,906,737**	**1,766,937**
what one transaction may spend	33,554,432 cycles	**10,000,000 gas**
share of it	**5.68%**	**17.67%**

The program is 7.3% cheaper and its budget share 3.1x worse: the framework fell (floor 2.8x, per-package handling 27x, borsh in place of RISC Zero serde words), and the denominator fell further. At five signers the shares are 9.39% and 29.34%; marginal cost per

signer stays flat, M0 §3.2′s non-amortisation result unchanged. The delivered product (sections 2–3) runs the same update at **411,376 gas**.

1.1 Disposition of M0′s recommendations

M0	recommendation	disposition
§5.1	accelerate secp256k1	stands, firmer — software recovery is 333% of the v0.3 limit, against 99.4% of M0′s budget
§5.1	leave keccak256 in software	**revised — the coprocessor is the delivered configuration**; the trade that favoured software inverted (section 4)
§5.2	keep the RFP heartbeat defaults	stands — cost still does not constrain cadence
§5.3	default to push, price pull	stands — both modes cost the same ≈1.77M-gas plan
§5.4	recommend a recovery precompile	**withdrawn — achieved in guest code** (section 2)

One sentence of M0 is withdrawn with §5.4: from §3.2′s 621,555 cycles per signer the delivered READMEs derived a ceiling of roughly **53 signers** in one transaction. Against 10,000,000 gas at 583,024 a signer it is about **17**; the product ships `MAX_SIGNERS` 12 (M5-00). The claim behind it — linear cost bounds price, not feasibility — survives.

2. Recovery: the delivered backend

M0 §3.2 established that recovery does not amortise and dominates the program; §4 sketched a recovery precompile at an 8x–9x reduction; §5.4 recommended asking Logos for one. The disposition is one line: **withdrawn — the saving is achieved in guest code, and requires nothing from Logos.**

The delivered backend, measured in the same in-zkVM harness as the path M0 priced (`methods/tests/recovery_cost.rs`):

backend	cycles per recovery
patched `k256` — the path M0 priced	568,350
delivered first-party backend	**109,502**

What the product pays per update, plan plus both applies, relayer trimming payloads to the threshold:

update	M0′s path	delivered
3 signers — the production threshold	1,766,937	**411,376**
5 signers	2,933,979	642,321

How it works. Scalar multiplication by GLV decomposition with wNAF and a fixed-base table, plus an addition-chain square root for point decompression. The field and curve arithmetic underneath is `risc0-crypto`'s bigint2 circuits; the orchestration is first-party, small and separately testable.

How it is tested. A 50-case in-zkVM equivalence suite holds the backend exactly to the pinned `k256` path, errors included: GLV sign splits, both `v` conventions, `r` and `s` at zero and at the curve order, decompression edges, a signature recovering to the identity.

The audit caveat, stated plainly. `risc0-crypto` is release-candidate grade and a different cryptographic implementation from the widely-reviewed `k256` — an audit question before a cost question. And changing the guest changes the image id, which is the program id, so every derived account address moves: the migration ADR 22 records, and the real cost of shipping this.

3. The round: multi-point packages

M0 priced the five-feed round as five independent submissions, one package set per feed. RedStone also publishes **multi-point packages** — one signature covering several data points — and the live __FAST__ package on `redstone-primary-prod` carries nine feeds including BTC, ETH and SOL (not XMR or ZEC), all five production signers, every 10-second round. The round therefore becomes one __FAST__ update settling three feeds, which share five recoveries instead of fifteen, plus two single-point updates (XMR, ZEC), priced exactly as in M0.

This is production practice, conformance-tested against captured production signatures (`verifier-core/tests/multipoint_fast.rs`, four passing tests): a captured round reconstructs byte-for-byte, every signature recovers to the published signer, and — the safety direction — a __FAST__ payload reaches no threshold for XMR, so a package cannot vouch for a feed it never priced.

Measured through the delivered guest on the captured __FAST__ payload itself (`methods/tests/phases.rs`, `a_multipoint_fast_submission_measures`):

submission, update, 3 signers	plan	applies	total
via __FAST__ (9 points)	413,779	15,652 + 12,072	**441,503**
single-point	383,652	15,652 + 12,072	411,376

Nine points cost **30,127** more than one — the span premium is noise once keccak256 rides the coprocessor. The round at the production threshold is 441,503 (plus roughly two more price applies, ≈24k, once the aggregator settles three feeds from one submission) + 411,376 (XMR) + 411,376 (ZEC) ≈ **1.29M gas — 12.9% of a block** — against 155% for the same round on M0′s path at five signers, a factor of about twelve. Under proof composition the 15 → 9 recovery reduction also shrinks the inner proof the relayer produces.

On the **storage market** (`kanon-chain/tests/storage_gas.rs`, `the_final_round_on_the_storage_market`) the nine-point submission is 2,506 storage gas against the single-point 970: the round is 2,506 + 2 × 970 = **4,446 storage gas — 0.44% of the 1,000,000 block cap, 0.89% of the 500,000 target**; the future three-feed settle adds ≈ +342 bytes, still under half a percent of the cap.

What remains to build is the aggregator settling several feeds from one submission — one plan, three applies. **The dependency, and the ask:** the composition of __FAST__ is RedStone's and can change under us, so a relayer using it must verify the needed points are present and fall back to single-point packages; the durable ask is one line in RedStone's node manifest — XMR and ZEC into the `multiPointPackages` map — demonstrably their routine practice.

4. The chain model M0 lacked

M0 §2.2 said public execution is capped in cycles and not proven, so cycles are the cost. On v0.3 cycles are the **fee**: gas is cycles one for one, against a 10,000,000 limit (`MAX_GAS_EXEC`) the transaction declares — a larger declaration is rejected (`FeeError::GasLimitAboveCap`) — with a base fee starting at 8 and moving ±12.5% per block against `TARGET_GAS_EXEC` of 5,000,000. A program is two metered runs, a `plan` plus one `apply` per shard; every figure in this addendum is the sum. Bytes are billed on a second market, one storage gas per serialized byte, against `TARGET_GAS_STOR` 500,000 and `MAX_GAS_STOR` 1,000,000 per block.

4.1 The per-block cap

`MAX_GAS_EXEC` is not only a per-transaction ceiling: `accumulate_exec_gas` sums each transaction's **actually charged** cycles across a block and rejects the block past the same 10,000,000 (`lez/chain_state/src/apply.rs`). The per-transaction and per-block caps being the same number is the unusual part — one transaction is permitted to consume an entire block. Chain-wide, across all programs and all users:

	gas	share of a whole block
one update — M0′s path, 3 signers	1,766,937	17.7%
one update — M0′s path, 5 signers	2,933,979	29.3%
one update — delivered, 3-of-5 threshold	411,376	4.1%
five-feed round — M0′s path, 5 signers	**14,669,895**	**147% — will not fit**
five-feed round — delivered, 3-of-5, multi-point (section 3)	**≈1,290,000**	**12.9%**

The two scopes are not comparable row to row: the first three price **one** feed's update, the last two price all **five** feeds settling in the same block. The delivered round is cheaper than one update on M0′s path because it changes three things at once: three signatures verified instead of five, a recovery at 109,502 cycles instead of 568,350, and BTC, ETH and SOL sharing one submission.

This is a bound on **simultaneity**, not on throughput: even on M0′s path the five feeds settle in two or three blocks, and the RFP's one-hour heartbeat leaves 3,600 seconds to place five updates. On the delivered round the bound is moot — the round fits in one block with most of it free. What the cap leaves worth stating: two five-signer updates on M0′s path alone exceed `TARGET_GAS_EXEC`, so coinciding updates exercise the base fee; and **one transaction may take a third of a block**, a property of the chain rather than of this adaptor, and the one reading we would like confirmed as intended (section 5).

Not measured: the block rate. Shipped sequencer configs set `block_create_timeout` to `10s` (`15s` in the debug profile) — an upper bound on how long a sequencer waits, not a fixed interval — and the public testnet returned `502` throughout the attempt to measure the live rate. It matters only for the ceiling: updates per unit time is roughly five per block times the block rate, and the RFP's cadence is orders of magnitude below any plausible value of either term.

4.2 Storage gas never binds

A 3-signer submission is **970 bytes** as captured — a signer costs a flat 162 bytes — measured where the envelope is built (`kanon-chain/tests/storage_gas.rs`) and pinned to be the byte string `classify` prices. **Execution binds 182 times sooner**: 17.67% of the per-block execution budget against 0.10% of the storage one, and still roughly 22 times sooner at the worst case the design permits (12 signers, payload padded to 32 KiB: 73.2% against 3.35%). Storage is 0.05% of a submission's own fee. The delivered round's storage footprint is in sections 1 and 3; no conclusion here rests on a storage figure.

4.3 One recommendation revised: M0 §5.1′s keccak256 choice

M0 recommended software keccak256 when keccak was 2.72% of an update and the coprocessor's only measured effect was a proving penalty — +23% time, +223,112 composite-proof bytes. In the delivered solution that trade is inverted: recovery shrank 5.2x, so keccak's share of an update grew, and nine-point spans cost **+356,646 gas** across five packages in software against **+48,776** on the coprocessor. **The coprocessor is the delivered configuration**, and every product figure in this addendum — 441,503, 411,376, the 12.9% round — is a coprocessor number.

The penalty M0 weighed is also smaller than it looked. It exists only in composite receipts, which LEZ's private path never puts on the wire: proved succinct — the only receipt the private path carries — both configurations are **223,789 bytes, identical to the byte**, so the coprocessor costs 0 bytes and +31.62 s (+19%); and the 6 GiB VRAM cliff does not reproduce on the same card in either proving mode. The choice has **no on-chain storage effect** — storage gas prices the serialized transaction envelope, which is independent of guest configuration — so the proving penalty matters only where the same image is proven, the private path (ADR 6′s caveat), and there it is wall-clock for whoever proves, not gas. Pinned by `the_coprocessor_penalty_does_not_survive_recursion`.

One planning note: a private transaction is charged a flat `PRIVATE_GAS_STOR` = 223,551 + 512 = 224,063 storage gas — 22% of a block's storage budget, marked provisional upstream. Re-pinning it at real proof sizes would not affect this disposition, because the coprocessor adds no bytes to the succinct receipt the chain carries.

5. Caveats, and asks

Extending M0 §6, what each new measurement is a statement about:

- **Proving was sampled at three signers.** Proof sizes are deterministic and reproduced to the byte; times and memory are not, so the wall-clock gap in section 4 is a direction rather than a ratio.
- **The recovery row in section 1 is a remainder**, from one subtraction whose subtrahend is itself measured. It agrees with `m0/cost-baseline/`'s independent figure and with the v0.2 remainder to within ten cycles.
- **The delivered recovery backend is an audit question before a cost question** (section 2): the arithmetic underneath is release-candidate `risc0-crypto`, and shipping it moves the image id and every derived address (ADR 22).
- **The `__FAST__` composition is RedStone's** and can change under us; a relayer using it must verify the needed points are present and fall back to single-point packages (section 3).
- **The storage figures are one envelope shape** — the submission the relayer and CLI send — with a linear, pinned per-signer cost; a statement about Kanon's submission, not LEZ transactions in general.

Extending M0 §7, the asks:

- **Of Logos: the per-block cap.** `MAX_GAS_EXEC` is both the per-transaction and the per-block execution cap (section 4), so a single transaction may lawfully fill an entire block. The delivered round does not come near that — 12.9%, in one transaction or five — so this is a question about the chain's posture rather than our need: is one-transaction-per-block intended, and should heavy users expect to spread work across blocks? Our reading is derived from source rather than documentation, which is why we ask.

- **Of Logos: the token's denomination.** It is not stated in LEZ, the wallet, or our repo, and gas × base fee only gives base units — three signers at the floor base fee of 8 is 14,135,496 units per update, 619 billion a year at 43,800 updates.

- **Of RedStone: one manifest line.** Add XMR and ZEC to a multi-point package via the `multiPointPackages` map in the node manifest (section 3). The live `__FAST__` package demonstrates this is their routine practice.

- **Nothing else.** M0 §5.4′s ask — a secp256k1 recovery precompile — is withdrawn: the saving is achieved in guest code, and whether to ship it is ours to decide (section 2).

6. Reproducing this

As M0 §8, with the v0.3 workspace and the product branch added:

```
nix-shell                                          # NixOS; else rustup + rzup

cd m0/v0.3_rerun
cargo test --release                               # the v0.3.0 figures, seconds
cargo test --release -- --ignored                  # proof sizes, minutes
cargo run  --release -p lez-probe                  # single-point figures
cargo run  --release -p lez-probe -- --multifeed # the 9-point spans (FINDINGS 6.1)

cd ../../kanon-chain
cargo test --release --test storage_gas            # the storage figures
```

The delivered-product figures live on the product repository:

test	what it asserts
`methods/tests/recovery_cost.rs`	recovery cost in isolation, and the 50-case equivalence suite (section 2)
`methods/tests/phases.rs`, `a_multipoint_fast_submission_measures`	section 3′s submission table
`verifier-core/tests/multipoint_fast.rs`	the `___FAST___` conformance suite (section 3)
`kanon-chain/tests/storage_gas.rs`, `the_final_round_on_the_storage_market`	the delivered round's 4,446 storage gas (sections 1 and 3)

Every figure is asserted by exact equality, in the style M0′s are. Assertions with no M0 counterpart: `the_cryptography_did_not_move_between_lez_generations` holds the verification cost against M0′s figures across the rewritten program model — its failure voids the comparison this document rests on; `a_submission_is_a_material_share_of_a_whole_block` pins section 4 as a shape, failing if LEZ raises the cap; `the_r0crypto_program_is_unchanged` and `the_irreducible_share_of_a_recovery_is_unchanged` pin section 2′s recovery measurements; `the_coprocessor_penalty_does_not_survive_recursion` pins section 4′s keccak256 re-measurement, and also fails if the coprocessor ever proves **faster**; `the_wire_bytes_are_what_the_chain_prices` pins that the envelope `kanon-chain` builds is the byte string `classify` prices.

The full working behind this addendum, including the intermediate measurements it omits, is `m0/v0.3_rerun/FINDINGS.md`; the asserted constants are in `m0/v0.3_rerun/lez-probe/host/tests/guardrails.rs` and `kanon-chain/tests/storage_gas.rs`, the unedited run output in `m0/v0.3_rerun/raw/`. The v0.3 workspace holds **every pin identical to `m0/`'s** (`m0/versions.md`) and changes only the LEZ revision, so differences between the two figure sets are attributable to LEZ alone.